

STGCN++ 및 RTMO 학습 설정 파일 가이드

목차

1. 개요
2. STGCN++ 설정 파일 분석
 - 모델 구조 설정
 - 데이터셋 설정
 - 파이프라인 설정
 - 학습 하이퍼파라미터
 - 손실 함수 및 최적화
3. RTMO 설정 파일 분석
 - 모델 아키텍처
 - 키폰트 변환 시스템
 - 데이터 파이프라인
 - 학습 전략
4. 키폰트 변환 과정 상세 가이드
5. 실전 튜닝 가이드
6. 문제 해결 가이드

개요

이 가이드는 STGCN++ (Spatial-Temporal Graph Convolutional Networks Plus Plus)와 RTMO (Real-Time Multi-Object) 모델의 학습 설정 파일에 대한 포괄적인 기술 문서입니다. 각 설정 항목의 이론적 배경부터 실전 튜닝 방법까지 상세히 다룹니다.

주요 특징

- **STGCN++**: 골격 기반 행동 인식을 위한 그래프 컨볼루션 신경망
- **RTMO**: 실시간 다중 객체 포즈 추정을 위한 원스테이지 모델
- **키폰트 변환**: 다양한 데이터셋의 키폰트를 COCO 17개 형식으로 통일

STGCN++ 설정 파일 분석

모델 구조 설정

1. Backbone 설정

```
model = dict(
    backbone=dict(
        type='STGCN',
        gcn_adaptive='init',
        gcn_with_res=True,
        tcn_type='mstcn',
        graph_cfg=dict(layout='coco', mode='spatial'),
        num_stages=4,
        base_channels=64,
        inflate_stages=[2],
        down_stages=[2],
    )
)
```

주요 파라미터 상세 분석:

gcn_adaptive (Graph Convolution 적응성)

- 'init': 초기화 시에만 그래프 구조 학습
- 'offset': 오프셋을 통한 적응적 그래프 구조
- 이론: 고정된 그래프 구조 대신 학습 가능한 그래프로 더 나은 특징 추출
- 튜닝 효과: 'offset'은 더 정확하지만 메모리 사용량 증가

gcn_with_res (잔차 연결)

- True: GCN 층에 잔차 연결 추가
- 이론: 깊은 네트워크에서 그래디언트 소실 문제 해결
- 성능 영향: 보통 2-5% 정확도 향상, 약간의 연산 오버헤드

tcn_type (Temporal Convolution 유형)

- 'mstcn': Multi-Scale Temporal Convolution
- 'unit_tcn': 단일 스케일 TCN
- 이론: 다중 시간 스케일로 단기/장기 시간적 의존성 포착
- 성능: mstcn이 보통 3-7% 더 우수하지만 파라미터 2배 증가

num_stages (네트워크 깊이)

- **설정값:** 4 (6에서 감소)
- **이론:** 각 스테이지는 공간-시간 블록의 집합
- **튜닝 가이드:**
 - 2-3: 빠른 추론, 단순한 동작에 적합
 - 4-5: 균형잡힌 성능 (권장)
 - 6-8: 복잡한 동작, 과적합 위험 증가

base_channels (기본 채널 수)

- **설정값:** 64 (48에서 증가)
- **이론:** 각 레이어의 특징 맵 채널 수 결정
- **메모리 영향:** 64 → 128로 증가 시 메모리 사용량 4배 증가
- **성능 트레이드오프:**
 - 32: 빠름, 정확도 낮음
 - 64: 균형잡힌 선택
 - 128: 높은 정확도, 높은 메모리 사용

inflate_stages (채널 확장 단계)

- **설정값:** [2] (단일 확장)
- **이론:** 지정된 스테이지에서 채널 수를 2배로 확장
- **계산:** stage 2에서 64 → 128 채널로 확장
- **튜닝 전략:**
 - [1, 3]: 초기와 후반에 확장 (복잡한 특징)
 - [2]: 중간 확장 (균형)
 - []: 확장 없음 (경량화)

2. Classification Head 설정

```
cls_head=dict(
    type='GCNHead',
    num_classes=2,
    in_channels=128,
    dropout=0.3,
    loss_cls=dict(
        type='CrossEntropyLoss',
        class_weight=[1.2, 1.0]
    )
)
```

주요 파라미터 분석:

dropout (드롭아웃 비율)

- **설정값:** 0.3 (0.5에서 감소)
- **이론:** 과적합 방지를 위한 정규화 기법
- **최적화 가이드:**
 - 0.1-0.2: 큰 데이터셋 (>10K 샘플)
 - 0.3-0.4: 중간 데이터셋 (1K-10K 샘플)
 - 0.5-0.7: 작은 데이터셋 (<1K 샘플)

class_weight (클래스 가중치)

- **설정값:** [1.2, 1.0] (NonFight:Fight)
- **이론:** 불균형 데이터셋에서 소수 클래스에 더 높은 가중치
- **계산 방법:** $\text{weight} = \text{total_samples} / (\text{num_classes} * \text{class_samples})$
- **효과:** 정밀도-재현율 균형 개선

데이터셋 설정

1. 데이터 경로 및 주석 파일

```
dataset_type = 'PoseDataset'
data_root = '/workspace/recognizer/test_data'
ann_file_train = '/workspace/recognizer/output/.../train.pkl'
ann_file_val = '/workspace/recognizer/output/.../val.pkl'
ann_file_test = '/workspace/recognizer/output/.../test.pkl'
```

데이터셋 구조 요구사항:

- **PoseDataset:** MMAAction2의 표준 포즈 데이터셋 형식
- **PKL 형식:** 피클 파일로 저장된 주석 데이터
- **필수 필드:** frame_dir, label, img_shape, original_shape, total_frames, keypoint, keypoint_score

파이프라인 설정

1. 학습 파이프라인

```
train_pipeline = [
    dict(type='PreNormalize2D'),
    dict(type='GenSkeFeat', dataset='coco', feats=['b']),
    dict(type='UniformSampleFrames', clip_len=100),
    dict(type='PoseDecode'),
    dict(type='FormatGCNInput', num_person=4),
    dict(type='PackActionInputs')
]
```

각 단계별 상세 분석:

PreNormalize2D

- **목적:** 키포인트 좌표를 이미지 크기로 정규화
- **수식:** $\text{normalized_coord} = \text{coord} / \text{image_size}$
- **중요성:** 다양한 해상도의 비디오에서 일관된 학습 가능

GenSkeFeat

- **dataset='coco':** COCO 17개 키포인트 형식 사용
- **feats=['b']:** Bone (뼈대) 특징만 사용
- **옵션:**
 - 'j': Joint (관절) 특징
 - 'b': Bone (뼈대) 특징
 - 'jm': Joint motion (관절 움직임)
 - 'bm': Bone motion (뼈대 움직임)
- **성능 비교:** Bone 특징이 행동 인식에서 보통 더 우수

UniformSampleFrames

- **clip_len=100:** 각 비디오 클립에서 100프레임 샘플링
- **이론:** 긴 비디오에서 고정된 길이의 시퀀스 추출
- **튜닝 가이드:**
 - 50-64: 빠른 동작, 메모리 절약
 - 100-128: 균형잡힌 선택 (권장)
 - 200+: 복잡하고 긴 동작

num_person=4

- **의미:** 최대 4명의 사람까지 추적
- **메모리 영향:** 사람 수에 비례하여 메모리 사용량 증가
- **최적화:** 실제 시나리오에 맞게 조정 (1-2명이면 2로 설정)

2. 데이터 증강 전략

추가 가능한 데이터 증강 (예시)

```
train_pipeline = [
    dict(type='PreNormalize2D'),
    dict(type='GenSkeFeat', dataset='coco', feats=['b']),
    dict(type='RandomScale', scale=0.1), # 스케일 증강
    dict(type='RandomRot', rot=0.1), # 회전 증강
    dict(type='UniformSampleFrames', clip_len=100),
    dict(type='PoseDecode'),
    dict(type='FormatGCNInput', num_person=4),
    dict(type='PackActionInputs')
]
```

학습 하이퍼파라미터

1. 배치 크기 및 워커 설정

```
train_dataloader = dict(
    batch_size=32, # 16에서 증가
    num_workers=8,
    persistent_workers=True,
    sampler=dict(type='DefaultSampler', shuffle=True),
)
```

배치 크기 최적화:

- **메모리 계산:** GPU메모리 $\geq$ batch_size * sequence_length * feature_dim * 4bytes
- **성능 영향:**
 - 8-16: 작은 GPU, 불안정한 학습
 - 32-64: 균형잡힌 선택 (권장)
 - 128+: 큰 GPU, 안정적 학습, 학습률 조정 필요

2. 옵티마이저 설정

```
optim_wrapper = dict(
    type='OptimWrapper',
    optimizer=dict(
        type='AdamW',
        lr=0.0001,          # 기본 학습률
        weight_decay=0.001, # L2 정규화
        betas=(0.9, 0.999),
        eps=1e-8
    ),
    clip_grad=dict(max_norm=2.0, norm_type=2)
)
```

AdamW 하이퍼파라미터:

lr (학습률)

- **설정값:** 0.0001
- **이론:** 그래디언트 업데이트 크기 조절
- **튜닝 전략:**
 - 0.00001: 안정적이지만 느린 수렴
 - 0.0001: 균형잡힌 선택
 - 0.001: 빠른 수렴, 불안정 위험

weight_decay (가중치 감쇠)

- **설정값:** 0.001
- **목적:** 과적합 방지를 위한 L2 정규화
- **최적값:** 데이터셋 크기에 반비례 (작은 데이터 = 높은 decay)

clip_grad (그래디언트 클리핑)

- **max_norm=2.0:** 그래디언트 노름의 최대값
- **목적:** 그래디언트 폭발 방지
- **조정:** 학습 불안정 시 1.0으로 감소

3. 학습률 스케줄러

```
param_scheduler = [
    dict(
        type='LinearLR',
        start_factor=0.1,
        by_epoch=True,
        begin=0,
        end=3,
        convert_to_iter_based=True
    ),
    dict(
        type='MultiStepLR',
        by_epoch=True,
        begin=3,
        milestones=[15, 25],
        gamma=0.5
    )
]
```

Warmup + MultiStep 전략:

LinearLR (Warmup)

- **목적:** 초기 학습 안정성 확보
- **start_factor=0.1:** 실제 학습률의 10%로 시작
- **end=3:** 3 에폭 동안 웜업

MultiStepLR

- **milestones=[15, 25]:** 15, 25 에폭에서 학습률 감소
- **gamma=0.5:** 각 마일스톤에서 학습률을 절반으로 감소
- **효과:** 점진적 수렴으로 더 나은 최적해 탐색

손실 함수 및 최적화

1. 교차 엔트로피 손실

```
loss_cls=dict(
    type='CrossEntropyLoss',
    class_weight=[1.2, 1.0] # NonFight:Fight 가중치
)
```

수식:

$$L = -\sum(w_i * y_i * \log(p_i))$$

여기서 w_i 는 클래스 가중치, y_i 는 실제 라벨, p_i 는 예측 확률

클래스 불균형 해결:

- **가중치 계산:** $w_i = \text{total_samples} / (\text{num_classes} * \text{class_i_samples})$
- **효과:** F1-Score 균형 개선

RTMO 설정 파일 분석

모델 아키텍처

1. 백본 네트워크

```
backbone=dict(
    type='CSPDarknet',
    deepen_factor=1.0,
    widen_factor=1.0,
    out_indices=(2, 3, 4),
    spp_kernel_sizes=(5, 9, 13),
    norm_cfg=dict(type='BN', momentum=0.03, eps=0.001),
    act_cfg=dict(type='Swish'),
)
```

CSPDarknet 구조 분석:

deepen_factor / widen_factor

- **deepen_factor:** 네트워크 깊이 조절 (1.0 = 기본)
- **widen_factor:** 채널 수 조절 (1.0 = 기본)
- **변형 모델:**
 - RTMO-S: deepen=0.33, widen=0.5
 - RTMO-M: deepen=0.67, widen=0.75
 - RTMO-L: deepen=1.0, widen=1.0

out_indices=(2, 3, 4)

- **의미:** Feature Pyramid Network를 위한 다중 스케일 특징
- **스케일:** 1/8, 1/16, 1/32 해상도의 특징 맵
- **용도:** 다양한 크기의 객체 탐지

SPP (Spatial Pyramid Pooling)

- **kernel_sizes=(5, 9, 13)**: 다중 스케일 풀링
- **목적**: 다양한 크기의 수용 영역으로 컨텍스트 정보 포착

2. 넥 네트워크

```
neck=dict(
    type='HybridEncoder',
    in_channels=[256, 512, 1024],
    hidden_dim=256,
    output_indices=[1, 2],
    encoder_cfg=dict(
        self_attn_cfg=dict(embed_dims=256, num_heads=8, dropout=0.0),
        ffn_cfg=dict(
            embed_dims=256,
            feedforward_channels=1024,
            ffn_drop=0.0,
            act_cfg=dict(type='GELU')
        )
    )
)
```

HybridEncoder 구조:

Self-Attention 메커니즘

- **num_heads=8**: 멀티헤드 어텐션의 헤드 수
- **embed_dims=256**: 임베딩 차원
- **수식**: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k})V$
- **효과**: 장거리 의존성 포착, 키포인트 간 관계 학습

Feed-Forward Network

- **feedforward_channels=1024**: FFN의 은닉층 크기
- **비율**: 일반적으로 embed_dims의 4배
- **활성화**: GELU (Gaussian Error Linear Unit)

3. 헤드 네트워크

```
head=dict(
    type='RTMOHead',
    num_keypoints=17,
    featmap_strides=(16, 32),
    head_module_cfg=dict(
        num_classes=1,
        in_channels=256,
        cls_feat_channels=256,
        channels_per_group=36,
        pose_vec_channels=512,
        widen_factor=1.0,
        stacked_convs=2,
    )
)
```

RTMOHead 구조 분석:

Multi-Scale Detection

- **featmap_strides=(16, 32)**: 다중 스케일 특징 맵
- **purpose**: 다양한 크기의 사람 탐지

DCC (Dynamic Coordinate Classification)

- **num_bins=(192, 256)**: X, Y 좌표 분류를 위한 빈 수
- **이론**: 회귀 대신 분류로 좌표 예측의 정확도 향상
- **장점**: 더 안정적인 학습, 경계 근처 정확도 개선

키포인트 변환 시스템

1. 데이터셋별 키포인트 매핑

```
# AIC → COCO 변환 예시
aic_coco = [
    (0, 6), # AIC의 0번 → COCO의 6번 (왼쪽 어깨)
    (1, 8), # AIC의 1번 → COCO의 8번 (왼쪽 팔꿈치)
    (2, 10), # AIC의 2번 → COCO의 10번 (왼쪽 손목)
    # ... 계속
]

dataset_aic = dict(
    type='AicDataset',
    pipeline=[
        dict(type='KeypointConverter', num_keypoints=17, mapping=aic_coco)
    ],
)
```

KeypointConverter 상세 동작:

변환 과정:

1. **원본 키포인트 로드**: 각 데이터셋의 고유 형식
2. **매핑 테이블 적용**: (source_idx, target_idx) 튜플 기반
3. **COCO 형식 생성**: 17개 키포인트로 통일
4. **누락된 키포인트 처리**: 매핑되지 않은 키포인트는 (0,0,0)으로 설정

COCO 17 키포인트 구조:

0: 코	1: 왼쪽 눈	2: 오른쪽 눈	3: 왼쪽 귀	4: 오른쪽 귀
5: 왼쪽 어깨	6: 오른쪽 어깨	7: 왼쪽 팔꿈치	8: 오른쪽 팔꿈치	
9: 왼쪽 손목	10: 오른쪽 손목	11: 왼쪽 엉덩이	12: 오른쪽 엉덩이	
13: 왼쪽 무릎	14: 오른쪽 무릎	15: 왼쪽 발목	16: 오른쪽 발목	

2. 다중 데이터셋 통합

```
train_dataset = dict(
    type='CombinedDataset',
    datasets=[
        dataset_coco,      # COCO 데이터셋
        dataset_aic,       # AIC 데이터셋
        dataset_crowdpose, # CrowdPose 데이터셋
        # ... 더 많은 데이터셋
    ],
    sample_ratio_factor=[1, 0.3, 0.5, 0.3, 0.3, 0.4, 0.3],
)
```

Sample Ratio Factor:

- **의미:** 각 데이터셋에서 샘플링할 비율
- **COCO=1.0:** 기준 데이터셋 (100% 사용)
- **AIC=0.3:** COCO 대비 30% 샘플링
- **목적:** 데이터셋 크기 불균형 해결

데이터 파이프라인

1. 2단계 학습 파이프라인

```
# Stage 1: Mosaic + MixUp 증강
train_pipeline_stage1 = [
    dict(type='LoadImage'),
    dict(type='Mosaic', img_scale=(640, 640)),
    dict(type='YOLOXMixUp'),
    dict(type='RandomFlip'),
    dict(type='GenerateTarget', encoder=codec),
    dict(type='PackPoseInputs'),
]

# Stage 2: 단순 증강
train_pipeline_stage2 = [
    dict(type='LoadImage'),
    dict(type='BottomupRandomAffine'),
    dict(type='RandomFlip'),
    dict(type='GenerateTarget', encoder=codec),
    dict(type='PackPoseInputs'),
]
```

2단계 학습 전략:

Stage 1 (첫 580 에폭)

- **Mosaic**: 4개 이미지를 조합하여 다양한 크기와 위치의 객체 학습
- **MixUp**: 두 이미지를 알파 블렌딩하여 일반화 성능 향상
- **목적**: 강한 증강으로 모델의 견고성 향상

Stage 2 (마지막 20 에폭)

- **단순 증강**: Mosaic/MixUp 제거, 정확한 위치 학습에 집중
- **목적**: 세밀한 키포인트 위치 정확도 향상

2. Bottom-up 증강 기법

```
dict(  
    type='BottomupRandomAffine',  
    input_size=(640, 640),  
    shift_factor=0.1,  
    rotate_factor=10,  
    scale_factor=(0.75, 1.0),  
    pad_val=114,  
    distribution='uniform',  
    transform_mode='perspective',  
)
```

Bottom-up 특화 증강:

- **shift_factor**: 이미지 이동 비율 (0.1 = 10%)
- **rotate_factor**: 회전 각도 (± 10 도)
- **scale_factor**: 스케일 범위 (0.75~1.0배)
- **perspective**: 원근 변환으로 더 다양한 시점 학습

학습 전략

1. 학습률 스케줄링

```
param_scheduler = [
    # Warmup (5 에폭)
    dict(type='QuadraticWarmupLR', begin=0, end=5),

    # 1차 Cosine Annealing (280 에폭)
    dict(type='CosineAnnealingLR', begin=5, T_max=280, eta_min=0.0002),

    # 학습률 증가 (1 에폭)
    dict(type='ConstantLR', factor=2.5, begin=280, end=281),

    # 2차 Cosine Annealing (300 에폭)
    dict(type='CosineAnnealingLR', begin=281, T_max=300, eta_min=0.0002),

    # 고정 학습률 (20 에폭)
    dict(type='ConstantLR', factor=1, begin=580, end=600),
]
```

복합 스케줄링 전략:

QuadraticWarmup

- **수식:** $lr = base_lr * (epoch/warmup_epochs)^2$
- **효과:** 선형 워밍업보다 더 부드러운 시작

Cosine Annealing

- **수식:** $lr = eta_min + (base_lr - eta_min) * (1 + \cos(\pi * T_cur / T_max)) / 2$
- **장점:** 주기적인 학습률 변화로 local minimum 탈출

2단계 Annealing

- **280 에폭:** 첫 번째 수렴
- **학습률 증가:** 새로운 최적해 탐색을 위한 재시작
- **300 에폭:** 두 번째 수렴으로 더 나은 해 발견

2. 동적 설정 변경

```
custom_hooks = [
    dict(
        type='RTMOModeSwitchHook',
        epoch_attributes={
            280: {
                'proxy_target_cc': True,
                'overlaps_power': 1.0,
                'loss_cls.loss_weight': 2.0,
                'loss_mle.loss_weight': 5.0,
                'loss_oks.loss_weight': 10.0
            },
        },
    ),
]
```

모드 전환 후:

- **280 에폭:** 모델 설정 동적 변경
- **proxy_target_cc:** Coordinate Classification 타겟 활성화
- **손실 가중치 조정:** 세밀한 위치 학습 강화

3. 손실 함수 구성

```
loss_cls=dict(type='VariFocalLoss', loss_weight=1.0),      # 분류 손실
loss_bbox=dict(type='IoULoss', loss_weight=5.0),          # 바운딩 박스 손실
loss_oks=dict(type='OKSLoss', loss_weight=30.0),           # 키포인트 손실
loss_vis=dict(type='BCELoss', loss_weight=1.0),            # 가시성 손실
loss_mle=dict(type='MLECCLoss', loss_weight=1e-2),         # 좌표 분류 손실
```

다중 손실 함수:

VariFocalLoss (분류)

- **목적:** 불균형한 foreground/background 처리
- **특징:** 어려운 샘플에 더 높은 가중치

OKSLoss (키포인트)

- **수식:** Object Keypoint Similarity 기반
- **가중치=30.0:** 키포인트 정확도를 가장 중요하게 처리

MLECCLoss (좌표 분류)

- **이론:** Maximum Likelihood Estimation for Coordinate Classification
- **효과:** 회귀 대신 분류로 더 정확한 좌표 예측

키포인트 변환 과정 상세 가이드

1. 변환 파이프라인 아키텍처

```
# 전체 변환 과정
def keypoint_conversion_pipeline(source_keypoints, source_format, target_format='coco')
    """
    키포인트 변환 파이프라인

    Args:
        source_keypoints: 원본 키포인트 (N, K, 3) - N개 사람, K개 키포인트, (x,y,v)
        source_format: 원본 데이터셋 형식 ('aic', 'mpii', 'crowdpose', 등)
        target_format: 목표 형식 (기본값: 'coco')

    Returns:
        target_keypoints: 변환된 키포인트 (N, 17, 3)
    """

    # 1단계: 매핑 테이블 로드
    mapping_table = get_mapping_table(source_format, target_format)

    # 2단계: 좌표 변환
    target_keypoints = apply_mapping(source_keypoints, mapping_table)

    # 3단계: 가시성 처리
    target_keypoints = process_visibility(target_keypoints)

    # 4단계: 품질 검증
    target_keypoints = validate_keypoints(target_keypoints)

    return target_keypoints
```

2. 데이터셋별 변환 상세

AIC (AI Challenger) → COCO 변환

AIC 14 키포인트 → COCO 17 키포인트

```
aic_keypoint_structure = {
    0: "오른쪽 어깨",      1: "오른쪽 팔꿈치",    2: "오른쪽 손목",
    3: "왼쪽 어깨",      4: "왼쪽 팔꿈치",    5: "왼쪽 손목",
    6: "오른쪽 엉덩이",  7: "오른쪽 무릎",    8: "오른쪽 발목",
    9: "왼쪽 엉덩이",  10: "왼쪽 무릎",    11: "왼쪽 발목",
    12: "머리 상단",     13: "목"
}
```

```
aic_to_coco_mapping = [
    (0, 6), # 오른쪽 어깨
    (1, 8), # 오른쪽 팔꿈치
    (2, 10), # 오른쪽 손목
    (3, 5), # 왼쪽 어깨
    (4, 7), # 왼쪽 팔꿈치
    (5, 9), # 왼쪽 손목
    (6, 12), # 오른쪽 엉덩이
    (7, 14), # 오른쪽 무릎
    (8, 16), # 오른쪽 발목
    (9, 11), # 왼쪽 엉덩이
    (10, 13), # 왼쪽 무릎
    (11, 15), # 왼쪽 발목
    # 12, 13 (머리, 목)은 COCO에 직접 대응되지 않아 제외
]
```

누락된 COCO 키포인트 처리

```
missing_coco_keypoints = [0, 1, 2, 3, 4] # 코, 눈들, 귀들
```

변환 과정 상세:

1. **직접 매핑:** AIC의 12개 키포인트를 COCO의 해당 위치로 복사
2. **얼굴 키포인트 추정:** 머리와 목 위치를 기반으로 대략적 위치 계산
3. **가시성 전파:** 원본 가시성 정보를 변환된 키포인트에 적용

MPII → COCO 변환

```
# MPII 16 키포인트 구조
mpii_keypoint_structure = {
    0: "오른쪽 발목",      1: "오른쪽 무릎",      2: "오른쪽 엉덩이",
    3: "왼쪽 엉덩이",      4: "왼쪽 무릎",      5: "왼쪽 발목",
    6: "골반",            7: "흉부",            8: "상부 목",
    9: "머리 상단",        10: "오른쪽 손목",      11: "오른쪽 팔꿈치",
    12: "오른쪽 어깨",     13: "왼쪽 어깨",       14: "왼쪽 팔꿈치",
    15: "왼쪽 손목"
}

# 특별한 처리가 필요한 부분
def mpii_to_coco_special_processing(mpii_kpts):
    """MPII의 특수한 키포인트 구조 처리"""

    # 골반과 흉부를 이용해 COCO의 엉덩이 키포인트 생성
    pelvis = mpii_kpts[6]      # 골반
    thorax = mpii_kpts[7]      # 흉부

    # COCO 엉덩이는 골반 위치와 유사
    left_hip = pelvis.copy()
    right_hip = pelvis.copy()

    # 어깨 간격을 이용해 엉덩이 간격 추정
    shoulder_width = distance(mpii_kpts[12], mpii_kpts[13])
    hip_width = shoulder_width * 0.8 # 일반적으로 어깨보다 좁음

    return left_hip, right_hip
```

CrowdPose → COCO 변환

CrowdPose는 COCO와 유사하지만 순서가 다름

crowdpose_to_coco_mapping = [

```
(0, 5), # 왼쪽 어깨
(1, 6), # 오른쪽 어깨
(2, 7), # 왼쪽 팔꿈치
(3, 8), # 오른쪽 팔꿈치
(4, 9), # 왼쪽 손목
(5, 10), # 오른쪽 손목
(6, 11), # 왼쪽 엉덩이
(7, 12), # 오른쪽 엉덩이
(8, 13), # 왼쪽 무릎
(9, 14), # 오른쪽 무릎
(10, 15), # 왼쪽 발목
(11, 16), # 오른쪽 발목
```

]

CrowdPose의 얼굴 키포인트는 COCO와 1:1 대응

0-4번은 그대로 복사 가능

3. 고급 변환 기술

키포인트 보간 및 추정

```
def estimate_missing_keypoints(keypoints, mapping_info):  
    """누락된 키포인트를 기하학적 관계를 이용해 추정"""  
  
    # 코 위치 추정 (눈들의 중점 + 오프셋)  
    if has_keypoints(keypoints, ['left_eye', 'right_eye']):  
        left_eye = keypoints[1]  
        right_eye = keypoints[2]  
        nose = estimate_nose_from_eyes(left_eye, right_eye)  
        keypoints[0] = nose  
  
    # 귀 위치 추정 (눈과 어깨의 관계 이용)  
    if has_keypoints(keypoints, ['left_eye', 'left_shoulder']):  
        left_ear = estimate_ear_from_eye_shoulder(  
            keypoints[1], keypoints[5]  
        )  
        keypoints[3] = left_ear  
  
    return keypoints  
  
def estimate_nose_from_eyes(left_eye, right_eye):  
    """눈 위치로부터 코 위치 추정"""  
    # 두 눈의 중점  
    eye_center = (left_eye + right_eye) / 2  
  
    # 코는 일반적으로 눈 중점에서 아래쪽으로 약간 이동  
    nose_offset = np.array([0, 0.1 * abs(left_eye[1] - right_eye[1])])  
    nose = eye_center + nose_offset  
  
    return nose
```

품질 검증 및 필터링

```
def validate_keypoint_quality(keypoints, quality_threshold=0.5):
    """변환된 키포인트의 품질 검증"""

    quality_scores = []

    # 1. 해부학적 일관성 검증
    anatomy_score = check_anatomical_consistency(keypoints)
    quality_scores.append(anatomy_score)

    # 2. 키포인트 간 거리 검증
    distance_score = check_keypoint_distances(keypoints)
    quality_scores.append(distance_score)

    # 3. 대칭성 검증 (좌우 키포인트)
    symmetry_score = check_left_right_symmetry(keypoints)
    quality_scores.append(symmetry_score)

    # 4. 가시성 일관성 검증
    visibility_score = check_visibility_consistency(keypoints)
    quality_scores.append(visibility_score)

    overall_quality = np.mean(quality_scores)

    return overall_quality > quality_threshold, overall_quality


def check_anatomical_consistency(keypoints):
    """해부학적 일관성 검증"""
    violations = 0
    total_checks = 0

    # 무릎이 엉덩이와 발목 사이에 있는지 확인
    for side in ['left', 'right']:
        hip_idx = get_keypoint_index(f'{side}_hip')
        knee_idx = get_keypoint_index(f'{side}_knee')
        ankle_idx = get_keypoint_index(f'{side}_ankle')

        if all_visible([keypoints[hip_idx], keypoints[knee_idx], keypoints[ankle_idx]])
            if not is_between_points(keypoints[knee_idx],
                                    keypoints[hip_idx],
                                    keypoints[ankle_idx]):
                violations += 1
            total_checks += 1
```

```
return 1.0 - (violations / max(total_checks, 1))
```

4. 변환 최적화 전략

배치 처리 최적화

```
def batch_keypoint_conversion(batch_keypoints, source_format, target_format='coco'):
    """배치 단위 키포인트 변환 최적화"""

    batch_size, num_persons, num_keypoints, coords = batch_keypoints.shape

    # 매핑 테이블을 한 번만 로드
    mapping_table = get_mapping_table(source_format, target_format)

    # 벡터화된 변환 적용
    target_batch = np.zeros((batch_size, num_persons, 17, coords))

    for source_idx, target_idx in mapping_table:
        target_batch[:, :, target_idx, :] = batch_keypoints[:, :, source_idx, :]

    # 배치 단위 품질 검증
    quality_mask = batch_validate_quality(target_batch)

    return target_batch, quality_mask
```

메모리 효율적 처리

```
def memory_efficient_conversion(large_dataset, source_format,
                               chunk_size=1000, target_format='coco'):
    """대용량 데이터셋의 메모리 효율적 변환"""

    total_samples = len(large_dataset)
    converted_data = []

    for start_idx in range(0, total_samples, chunk_size):
        end_idx = min(start_idx + chunk_size, total_samples)
        chunk = large_dataset[start_idx:end_idx]

        # 청크 단위 변환
        converted_chunk = batch_keypoint_conversion(
            chunk, source_format, target_format
        )

        converted_data.append(converted_chunk)

        # 메모리 정리
        del chunk
        gc.collect()

    return np.concatenate(converted_data, axis=0)
```


실전 튜닝 가이드

1. 성능 분석 및 진단

학습 곡선 분석

```
def analyze_training_curves(log_file):  
    """학습 로그 분석을 통한 문제 진단"""  
  
    logs = parse_training_log(log_file)  
  
    # 손실 추세 분석  
    train_loss = logs['train_loss']  
    val_loss = logs['val_loss']  
  
    # 과적합 감지  
    overfitting_point = detect_overfitting(train_loss, val_loss)  
    if overfitting_point:  
        print(f"과적합 시작 지점: {overfitting_point} 에폭")  
        print("권장 조치: 정규화 강화, 데이터 증강 추가")  
  
    # 학습률 적정성 분석  
    lr_analysis = analyze_learning_rate(train_loss, logs['lr'])  
    if lr_analysis['too_high']:  
        print("학습률이 너무 높음: 발산하는 손실")  
        print("권장 조치: 학습률을 1/10로 감소")  
    elif lr_analysis['too_low']:  
        print("학습률이 너무 낮음: 매우 느린 수렴")  
        print("권장 조치: 학습률을 3-5배 증가")  
  
    return logs
```

모델 복잡도 vs 성능 트레이드오프

STGCN++ 모델 크기별 성능 가이드

```
model_configs = {  
    'lightweight': {  
        'num_stages': 3,  
        'base_channels': 32,  
        'inflate_stages': [],  
        'performance': '85% 정확도, 2x 빠름',  
        'use_case': '실시간 애플리케이션'  
    },  
    'balanced': {  
        'num_stages': 4,  
        'base_channels': 64,  
        'inflate_stages': [2],  
        'performance': '90% 정확도, 표준 속도',  
        'use_case': '일반적인 용도 (권장)'  
    },  
    'high_accuracy': {  
        'num_stages': 6,  
        'base_channels': 128,  
        'inflate_stages': [2, 4],  
        'performance': '93% 정확도, 2x 느림',  
        'use_case': '높은 정확도가 필요한 경우'  
    }  
}
```

2. 데이터별 튜닝 전략

작은 데이터셋 (<1K 샘플)

```
small_dataset_config = {
    'data_augmentation': {
        'rotation_range': 15,          # 더 강한 회전 증강
        'scale_range': (0.8, 1.2),    # 더 넓은 스케일 범위
        'noise_factor': 0.1,          # 노이즈 추가
    },
    'training': {
        'batch_size': 8,              # 작은 배치
        'dropout': 0.6,               # 높은 드롭아웃
        'weight_decay': 0.01,         # 강한 정규화
        'early_stopping': 10,         # 빠른 조기 종료
    },
    'model': {
        'num_stages': 3,              # 단순한 모델
        'base_channels': 32,          # 적은 채널
    }
}
```

중간 데이터셋 (1K-10K 샘플)

```
medium_dataset_config = {
    'data_augmentation': {
        'rotation_range': 10,
        'scale_range': (0.9, 1.1),
        'mixup_alpha': 0.2,           # MixUp 증강 추가
    },
    'training': {
        'batch_size': 32,
        'dropout': 0.3,
        'weight_decay': 0.001,
        'lr_schedule': 'cosine',      # 코사인 스케줄링
    },
    'model': {
        'num_stages': 4,              # 균형잡힌 모델
        'base_channels': 64,
    }
}
```

큰 데이터셋 (>10K 샘플)

```
large_dataset_config = {  
    'data_augmentation': {  
        'rotation_range': 5,          # 약한 증강  
        'scale_range': (0.95, 1.05),  
        'cutmix_prob': 0.5,          # CutMix 증강  
    },  
    'training': {  
        'batch_size': 64,            # 큰 배치  
        'dropout': 0.1,              # 낮은 드롭아웃  
        'weight_decay': 0.0001,      # 약한 정규화  
        'label_smoothing': 0.1,      # 라벨 스무딩  
    },  
    'model': {  
        'num_stages': 6,             # 복잡한 모델  
        'base_channels': 128,        # 많은 채널  
    }  
}
```

3. 하드웨어별 최적화

GPU 메모리 최적화

```
def optimize_for_gpu_memory(available_memory_gb):  
    """GPU 메모리에 따른 설정 최적화"""  
  
    if available_memory_gb < 8:  
        return {  
            'batch_size': 16,  
            'base_channels': 32,  
            'num_person': 2,  
            'clip_len': 50,  
            'gradient_accumulation': 2, # 그래디언트 누적  
        }  
    elif available_memory_gb < 16:  
        return {  
            'batch_size': 32,  
            'base_channels': 64,  
            'num_person': 4,  
            'clip_len': 100,  
            'mixed_precision': True, # 혼합 정밀도  
        }  
    else:  
        return {  
            'batch_size': 64,  
            'base_channels': 128,  
            'num_person': 6,  
            'clip_len': 200,  
            'channels_last': True, # 메모리 효율적 형식  
        }
```

다중 GPU 학습 최적화

```
# 분산 학습 설정
distributed_config = {
    'num_gpus': 4,
    'batch_size_per_gpu': 16,      # 총 배치: 64
    'sync_bn': True,               # 배치 정규화 동기화
    'find_unused_parameters': False, # 성능 최적화
    'gradient_compression': True,   # 통신 최적화
}

# DDP 최적화 설정
ddp_config = {
    'backend': 'nccl',
    'init_method': 'env://',
    'world_size': 4,
    'rank': 0,
    'gpu_id': 0,
    'bucket_cap_mb': 25,           # 통신 버킷 크기
    'broadcast_buffers': False,    # 불필요한 브로드캐스트 제거
}
```

문제 해결 가이드

1. 일반적인 학습 문제

수렴하지 않는 문제

증상: 손실이 감소하지 않거나 NaN이 발생

원인 및 해결책:

```
# 학습률 문제
if loss_exploding:
    new_lr = current_lr * 0.1
    optim_wrapper.optimizer.param_groups[0]['lr'] = new_lr

# 그래디언트 클리핑 강화
if gradient_exploding:
    clip_grad.max_norm = 1.0 # 2.0에서 감소

# 배치 정규화 문제
if batch_norm_issue:
    # 더 작은 모멘텀 사용
    norm_cfg = dict(type='BN', momentum=0.01, eps=0.001)
```

과적합 문제

증상: 훈련 정확도는 높지만 검증 정확도가 낮음

해결 전략:

```
# 정규화 강화
regularization_config = {
    'dropout': 0.5, # 기본 0.3에서 증가
    'weight_decay': 0.01, # 기본 0.001에서 증가
    'label_smoothing': 0.1, # 라벨 스무딩 추가
    'mixup_alpha': 0.2, # MixUp 증강
}

# 데이터 증강 강화
stronger_augmentation = {
    'rotation_range': 20, # 회전 범위 확대
    'scale_jitter': (0.8, 1.2), # 스케일 변화 확대
    'color_jitter': 0.3, # 색상 변화 추가
}

# 모델 단순화
simplified_model = {
    'num_stages': 3, # 6에서 3으로 감소
    'base_channels': 32, # 64에서 32로 감소
}
```

느린 수렴 문제

증상: 매우 느린 학습 진행

해결 방법:

```
# 학습률 증가
faster_training = {
    'lr': 0.001,          # 기본 0.0001에서 10배 증가
    'warmup_epochs': 1,   # 워업 기간 단축
    'cosine_restart': True, # 코사인 재시작
}

# 배치 크기 증가
larger_batch = {
    'batch_size': 64,      # 32에서 증가
    'accumulate_grad_batches': 2, # 효과적인 배치 크기 128
}

# 모델 초기화 개선
better_init = {
    'init_cfg': dict(type='Xavier', distribution='uniform'),
    'pretrained': 'path/to/pretrained/model.pth',
}
```

2. 키포인트 변환 문제

변환 품질 저하

문제: 변환된 키포인트가 부정확

진단 및 해결:


```
def diagnose_conversion_quality(original_kpts, converted_kpts):
    """변환 품질 진단"""

    # 1. 매핑 정확성 확인
    mapping_accuracy = check_mapping_accuracy(original_kpts, converted_kpts)

    # 2. 해부학적 일관성 확인
    anatomy_score = check_anatomical_consistency(converted_kpts)

    # 3. 가시성 전파 확인
    visibility_score = check_visibility_propagation(original_kpts, converted_kpts)

    if mapping_accuracy < 0.8:
        print("매핑 테이블 검증 필요")
        suggest_mapping_fixes()

    if anatomy_score < 0.7:
        print("해부학적 일관성 문제 - 후처리 필터링 추가")
        apply_anatomical_constraints()

    if visibility_score < 0.9:
        print("가시성 전파 문제 - 가시성 규칙 검토")
        fix_visibility_rules()
```

누락된 키포인트 처리

```
def handle_missing_keypoints(keypoints, strategy='interpolation'):
    """누락된 키포인트 처리 전략"""

    if strategy == 'interpolation':
        # 인접 키포인트 보간
        return interpolate_missing_keypoints(keypoints)

    elif strategy == 'anatomical_estimation':
        # 해부학적 관계 기반 추정
        return estimate_from_anatomy(keypoints)

    elif strategy == 'zero_padding':
        # 0으로 패딩
        return zero_pad_missing(keypoints)

    elif strategy == 'drop_sample':
        # 샘플 제거
        return None
```

3. 성능 최적화 문제

추론 속도 최적화

```
# 모델 경량화
lightweight_optimizations = {
    'model_pruning': {
        'sparsity': 0.3,          # 30% 가중치 제거
        'structured': True,      # 구조적 프루닝
    },
    'quantization': {
        'mode': 'int8',          # 8비트 양자화
        'calibration_samples': 100,
    },
    'knowledge_distillation': {
        'teacher_model': 'large_model.pth',
        'temperature': 4.0,
        'alpha': 0.7,
    }
}

# 추론 최적화
inference_optimizations = {
    'batch_inference': True,    # 배치 추론
    'tensorrt_optimization': True, # TensorRT 최적화
    'onnx_export': True,        # ONNX 변환
    'half_precision': True,     # FP16 사용
}
```

메모리 사용량 최적화

```
# 메모리 효율적 학습
memory_optimizations = {
    'gradient_checkpointing': True,    # 그래디언트 체크포인팅
    'cpu_offload': True,              # CPU 오프로드
    'mixed_precision': 'fp16',        # 혼합 정밀도
    'activation_checkpointing': True,  # 활성화 체크포인팅

    # 데이터 로더 최적화
    'pin_memory': True,
    'persistent_workers': True,
    'prefetch_factor': 2,
}

# 동적 배치 크기
def adaptive_batch_size(gpu_memory_usage):
    """GPU 메모리 사용량에 따른 동적 배치 크기 조정"""
    if gpu_memory_usage > 0.9:
        return current_batch_size // 2
    elif gpu_memory_usage < 0.6:
        return min(current_batch_size * 2, max_batch_size)
    else:
        return current_batch_size
```

이 가이드를 통해 STGCN++ 및 RTMO 모델의 설정 파일을 완전히 이해하고, 효과적으로 튜닝하여 최적의 성능을 달성할 수 있습니다. 각 설정의 이론적 배경을 이해하고, 실제 상황에 맞게 적절히 조정하는 것이 성공적인 모델 학습의 핵심입니다.